

Master Team Project Fall 2024

FindFul

Team 1

Member Name	Role
Sarah Majeed (sarah.majeed@informatik.hs-fulda.de)	Team Lead and Document Master
Samrat Bikram Shah	Backend Lead and Github Master
Annika Kristin Kaul	Frontend Lead
Kemer Jimenez	Team Member
Abdul Rahman Zuhaib	Team Member

Milestone 1

13/11/2024

Date Submitted	13/11/2024
Date Revised	13/11/2024

1. Executive Summary:

In the complex landscape of student housing, international students face significant challenges when searching for affordable, convenient, and suitable accommodations. Findful emerges as an innovative digital platform designed to transform the student rental experience in Fulda, Germany, addressing critical pain points for both students and landlords.

International and local students encounter numerous obstacles when seeking suitable housing: language barriers, limited local knowledge, complex rental processes, and the overwhelming task of finding affordable, quality accommodations. Traditional housing platforms are often generic, and do not cater specifically to student needs. Findful bridges this critical gap by creating a user-friendly, comprehensive housing marketplace tailored exclusively to students' unique requirements.

Our platform offers a holistic solution that goes beyond a simple display of property listings. By providing verified student and landlord profiles, comprehensive map-based property listings, and an integrated messaging system, Findful simplifies the entire housing search process. One of our standout features is the ability for students to apply for shared accommodation as a group, rather than as individuals. This is particularly beneficial in Fulda, where shared living arrangements are common among students. This feature allows students in Fulda to easily find and apply for shared housing, and also enables them to showcase their profiles to landlords, making it easier to find the perfect match. Overall, FindFul will facilitate students in exploring detailed property information, utilizing advanced search and filtering capabilities, and creating detailed profiles that help landlords understand their background, study details, and personal characteristics, making it easier for landlords to find suitable tenants.

The market potential for Findful is significant, driven by increasing international student enrollment and the growing demand for transparent, accessible housing solutions. Our platform represents more than just a technological solution – it is a scalable model that can potentially transform student housing searches in university cities across Germany and beyond.

We are a passionate group of software development students at Fulda University, united by a shared vision to solve a critical challenge faced by students. Comprised of five talented individuals from different academic and cultural backgrounds, each team member brings unique strengths to the project: some have experienced the challenges of international student housing, while others contribute deep technical skills in software development, user experience design, and system architecture. With a mix of technical knowledge, entrepreneurial thinking, and a deep understanding of student needs, our team is committed to transforming the student housing search experience.

2. Personae and Main Use cases:

Persona 1: Ram (International Student)

Background:

Ram is an incoming master's student from Nepal who has recently enrolled in an English-language business master's program in Fulda, Germany. He comes from a very modest economic background. He doesn't know German very well, nor does he know anyone in Germany. Ram is introverted and reserved, with basic technological skills.

Goals:

- He requires secure, affordable, and furnished accommodation near the university before classes begin in April.
- He wants to navigate the housing market in a foreign country with minimal local connections and language barriers.

Skills:

- He has ideas on basic internet navigation and the usage of standard web applications. However, he has limited experience with online housing platforms.

Pain Points:

- Having tight budget constraints makes it challenging for him to find affordable housing.
- Understanding and navigating German housing websites is difficult due to language barriers and a lack of local connections in Germany.
- He is unable to transport his personal furniture, requiring him to find a furnished accommodation.

Persona 2: Tim (Guest)

Background:

Tim is a German resident from Hessen, living in his own home near Frankfurt with his wife and two children. His daughter, Lily, is about to finish her high school and is considering attending university in Fulda. Lily wants to study in a smaller, quieter city compared to Frankfurt and finds Fulda to be a good option for a peaceful academic environment. He knows it's not practical to travel back and forth from his home to the university almost everyday for the classes.

Goals:

- Find a high-quality, spacious single-room apartment for his daughter near the university in Fulda.
- Ensure the accommodation is safe, comfortable, and close to campus to avoid long commutes.
- He doesn't care about the price of the room nor the furnishings.

Skills:

- Proficient in using technology and familiar with online real estate platforms.

Pain Points:

- Fulda is not confirmed as Lily's final choice yet, but he wants to prepare by scouting suitable accommodations.
- Since price and furnishings are not concerns, he finds it challenging to identify apartments that offer the right amount of space and comfort.
- He is unable to transport his personal furniture, requiring him to find a furnished accommodation.

Persona 3: Thomas Hartmann (Landlord)

Background:

Thomas is a real estate owner and landlord from Gießen, Germany. Together with his wife and daughter, he lives in a small house in the suburbs of Gießen. He specializes in renovating and renting out apartments to students in Hessen. Since 2005, Thomas has bought and renovated three apartment complexes in Fulda. He has recently finished renovating a building near the Fulda University of Applied Sciences.

Goals:

- Rent out all his apartments to students in Hessen.
- Plans to expand his business by buying and renovating new buildings to rent to students.
- Earn money for his family to provide them with a good life.

Skills:

- Good at constructing and renovating apartments.
- Basic understanding of technology and online real estate platforms.

Pain Points:

- Does not have a lot of time, as he has to drive across all of Hessen daily to maintain his multiple apartment complexes.
- New technologies overwhelm him. He prefers simple structures that allow him to reach his goals efficiently.

Persona 4: Anna Kaiser (Moderator)

Background:

Anna is a young entrepreneur from Wächtersbach, near Fulda, Germany. After her studies in International Business Management, she started working as a marketing manager and moderator at FindFul. She is an energetic, outgoing person who is passionate about her work. FindFul is not only her job; it is her passion project, as she has many international friends who had problems finding accommodations during their studies. When she helped her friends find accommodations, she realized that there were too many unserious and unhelpful listings on the market, which is why she decided to address that issue herself.

Goals:

- Help international students with an easy process to find suitable accommodations..
- Reduce poor listings of accommodations on the market.

Skills:

- Anna is very proficient in utilizing software and online tools. She has strong interpersonal skills and a good understanding of human behavior.

Pain Points:

- She doesn't like to use outdated web applications.
- She is frustrated by internet scams and unverified listings that waste her time and her friends' time when searching for accommodations.

Use Cases:

- 1. Search for Housing - Tim (Guest):** Tim opens the FindFul platform as a guest to search for affordable apartments. He filters the listings, specifying that the room should be a single-room apartment, over 25 square meters in size, and in close proximity to the university. Tim finds several options that meet his criteria, including both furnished and unfurnished choices. He is pleased to see that all the rooms are spacious, bright, and close to the university, giving him confidence that there are really good options available.
- 2. View Selected Housing - Ram (Student):** After searching for apartments within his budget of 380 euros, Ram opens two accommodations closest to the university in new tabs to compare them. As he examines each listing, he reviews detailed information about each room, including its location, existing tenants, and additional photos. He notices that one room is furnished, while the other is not, which helps him make a decision on which room to choose.
- 3. Connect with Landlords - Ram (Student):** After identifying a suitable furnished room from the list of rooms, Ram uses the platform's messaging feature to contact the landlord directly. He inquires about the availability, terms, and any additional costs.
- 4. Apply as a Group - Ram (Student):** Ram is about to move to Fulda with two friends. Ideally, they want to apply for a room together, as having familiar faces when moving to a new country can be helpful. On FindFul, after finding a suitable room for three people, they create a group of three to apply together. They then apply for the room as a group rather than individually and communicate with the landlord as a group.
- 5. Add Listing of New Real Estate - Thomas (Landlord):** Thomas has recently finished renovating a new apartment complex in Fulda. He now visits the portal to add a listing for the new building. He inserts the basic information about the building, details about the different apartments available, and adds photos to the portal. After a few steps, Thomas successfully submits the new listing.
- 6. Reviewing New Listings - Anna (Moderator):** Anna begins her daily review of new listings of real estates that should be posted to FindFul. She goes through the list one by one. She proofreads and inspects the submitted texts and images of the first listing and approves it. Anna continues with the second listing. The listing shows inconsistent images, so Anna declines the posting of the listing.

3. List of main data items and entities:

1. **Student** represents a verified Fulda University student who must complete email verification (hs-fulda.de) before registration.
2. **Landlord** is any individual or organization offering rental properties on the platform.
3. **Guest** is an unauthenticated visitor with limited platform access. They can browse public listings, view basic property information, and see location details, but cannot contact landlords or access full listing features.
4. **Moderator** is a system maintainer with comprehensive moderation powers. They review all new property listings and associated media, with the authority to approve or reject content based on platform guidelines.
5. **Real estate** is a representation of physical property available for rental, specifically targeting student accommodation in Fulda.
6. **Rental Listing** is the central data element representing a property available for rent. It contains all property details including title, description, monthly rent, and property type.
7. **Location** is a geographical description of a property's position, showing its location relative to important places like the university and city center.
8. **Media Item** is visual or image content associated with a rental listing, serving as a critical representation of the property's physical appearance and characteristics.

4. Initial list of functional requirements:

User registration and authentication

1. Users can register for an account as either a student or landlord. Before registration, users have guest status on the platform.
2. Users can log into their accounts using passkeys, offering a secure and convenient alternative to traditional passwords.
3. Users who are registering as a student are required to verify their student status to complete registration.
4. The system allows registered users to log out of their accounts.

Profile Management

5. Students can manage their profile including personal information, contact details, and upload profile photos.

Managing Listings

6. Landlords can post detailed accommodation listings with information such as rent, room type, amenities, location, and availability.
7. Each listing can include multiple media files for a clear visual presentation.
8. Landlords can edit or remove their accommodation listings as necessary.

9. Landlords can view student cards for any listing to which students have applied, providing a comprehensive idea regarding the students.

Admin functions

10. Admins have the authority to approve or reject listings based on their relevance. Listings will only be visible to students after receiving admin approval.
11. Admins can ban users who violate the platform's terms of service.

Search & Filtering

12. All Users on the system can perform searches for listings using keywords.
13. The system provides filters for listings, including key criteria such as their proximity to Hochschule or Bahnhof.
14. Users can explore multiple listings through an interactive map which allows them to access listing details directly from the map.

Accommodation Listings

15. Users can click on a listing to view detailed information including a description, photos, location on a map, rent breakdown and availability.
16. The system may suggest additional listings that match the user's previous search preferences or saved listings.
17. Users may be able to compare multiple listings side-by-side by key categories, such as rent, amenities, and location.

Group Application for Shared Accommodation

18. Students can make a group by adding up to 6 other students on the system.
19. Students can apply as a group to any rental accommodation that has available rooms for students in a group.

Communication

20. Students can contact landlords directly using in-app messaging.
21. Students and Landlords can manage their active chats in the My Messages section, for viewing conversations with each other.

5. List of non-functional requirements:

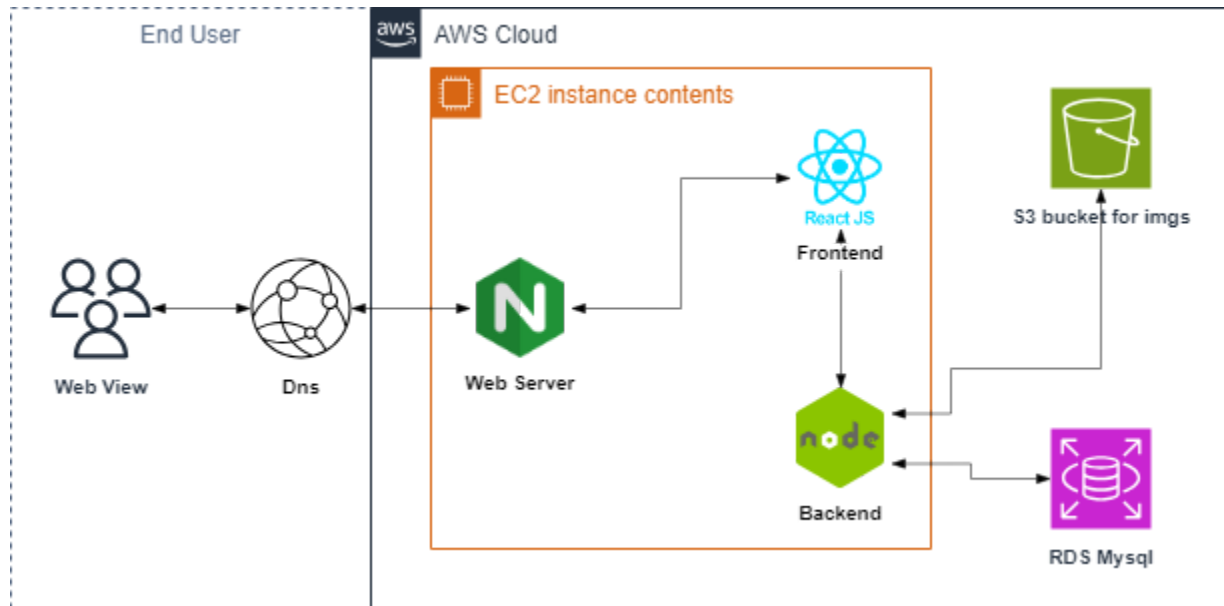
1. Application shall be developed, tested and deployed using tools and servers approved by Class CTO and as agreed in Milestone 0. Application delivery shall be from chosen cloud server
2. Application shall be optimized for standard desktop/laptop browsers e.g. must render correctly on the two latest versions of two major browsers
3. All or selected application functions must render well on mobile devices
4. Data shall be stored in the database on the team's deployment cloud server.
5. Full resolution free media shall be downloadable directly, and full resolution media for selling shall be obtained after contacting the seller/owner
6. No more than 50 concurrent users shall be accessing the application at any time
7. Privacy of users shall be protected, and all privacy policies will be appropriately communicated to the users.
8. The language used shall be English (no localization needed)
9. Application shall be very easy to use and intuitive
10. Application should follow established architecture patterns
11. Application code and its repository shall be easy to inspect and maintain
12. Google analytics shall be used (optional for Fulda teams)
13. No e-mail clients shall be allowed.
14. Pay functionality, if any (e.g. paying for goods and services) shall not be implemented nor simulated in UI.
15. Site security: basic best practices shall be applied (as covered in the class) for main data items
16. Application shall be media rich (images, video etc.). Media formats shall be standard as used in the market today
17. Modern SE processes and practices shall be used as specified in the class, including collaborative and continuous SW development
18. For code development and management, as well as documentation like formal milestones required in the class, each team shall use their own GitHub to be set-up by class instructors and started by each team during Milestone 0
19. The application UI (WWW and mobile) shall prominently display the following exact text on all pages "Fulda University of Applied Sciences Software Engineering Project, Fall 2024 For Demonstration Only" at the top of the WWW page. (Important to not confuse this with a real application).

6. Competitive analysis:

Key Feature	Wg-gesucht https://www.wg-gesucht.de/en/	Airbnb https://www.airbnb.com/	Immobilien Scout24.de https://www.immobilienscout24.de/	FindFul
Main Navigation via search	✓	✓	✓	✓
Focused on students	✓	✗	✗	✓
Embedded map	✓	✓	✓	✓
In-site messaging	✓	✓	✓	✓
Shareable student information	✗	✗	✗	✓
English support	✓	✓	✗	✓
Apply as a student group for a shared accommodation	✗	✗	✗	✓

Our application sets itself apart by allowing students to apply for rentals as groups and even find other students interested in forming groups for shared accommodation. In addition, our platform incorporates widely-used tools like map visualization combined with student-focused functionalities. These include full English support tailored for international students, along with comprehensive profiles on landlords and rental properties. Students can also create shareable profiles, giving landlords deeper insights into potential tenants and helping facilitate personality-based matches. With these features, we aim to create a highly personalized, student-friendly rental experience that makes finding suitable housing smoother, more collaborative, and better aligned with students' needs and preferences.

7. High-level system architecture and technologies used:



For the realization of this project, the proposed architecture is the following:

- **Frontend:** All the views from the app are developed in Javascript with React as the main framework.
- **Backend:** In charge of handling users and other services requests. Developed in node js with express.js as a backend framework.
- **Web Server:** Set up with Nginx, this module works as a gateway to communicate the frontend with the backend.
- **Cloud services:** We chose AWS as the project's cloud provider, mainly for the computer engine services, object storage for images and database simplification. The main services in use are:
 - **Ec2:** As the compute component, aws offers 1 vCpu and 1 Gb of ram for 750 hrs/month. It includes a dedicated ipv4 and 30 gbs of storage.
 - **S3:** S3 buckets help simplify the process of storing images and videos.
 - **RDS:** As with the S3, RDS simplify the set up and operation of our Mysql databases.

8.Team and roles:

Member Name	Role
Sarah Majeed	Team Lead and Document Master
Samrat Bikram Shah	Backend Lead and Github Master
Annika Kristin Kaul	Frontend Lead
Kemer Jimenez	Team Member
Abdul Rahman Zuhaib	Team Member

9. Checklist

Task	Status
Team found a time slot to meet (online) outside of the class	DONE
GitHub master chosen	DONE
Team decided and agreed together on using the listed SW tools and deployment server	DONE
Team ready and able to use the chosen back and front end frameworks and those who need to learn are working on learning and practicing	ON TRACK
Team lead ensured that all team members read the final M1 and agree/understand it before submission	DONE
GitHub organized as discussed in class (e.g. master branch, development branch, folder for milestone documents etc.)	ON TRACK

Master Team Project WS2024

FindFul

Team 1

Member Name	Role
Sarah Majeed (sarah.majeed@informatik.hs-fulda.de)	Team Lead and Document Master
Samrat Bikram Shah	Backend Lead and Github Master
Annika Kristin Kaul	Frontend Lead
Kemer Jimenez	Team Member
Abdul Rahman Zuhaib	Team Member

Milestone 2

27/11/2024

Date Submitted	27/11/2024
----------------	------------

1. Functional Requirements - prioritized:

Priority 1:

User registration and authentication

1. The system shall allow students and landlords to create an account.
2. The system shall allow users to log into their accounts using a unique email and a password combination and upon successful login, the user should be redirected to the homepage.
3. The system must verify the student's enrollment status at the time of registration by confirming that the student email ends with the domain "@informatik.hs-fulda.de," and if the email matches the domain, the system shall confirm the student to be verified.
4. The system shall allow registered users to log out and upon successful logout, the system shall be redirected to the login page.

Profile Management

5. The system shall allow registered users to manage their profiles, including personal information, contact details, and profile picture.

Manage Listings

6. The system shall allow the landlords to post detailed accommodation listings that include essential information such as:
 - Property description
 - Location
 - Rent
 - Amenities
 - Availability
7. The system shall allow landlords to include multiple media files, e.g., images for visual clarity.
8. The system shall allow landlords to edit or remove their accommodation listings.

Moderator Functions

10. The system shall allow the Moderator to approve or reject listings based on their content. Listings will only be visible to students after receiving moderator approval.
11. The system shall allow the Moderator to delete users from the system, who violate the platform's terms of service.

Search & Filtering

12. The system shall allow Users (unregistered and registered) to perform searches for listings using street information such as address, and postal code.
13. The system shall provide filters for searching listings, including:
 - Size of the apartment.
 - Type of the apartment i.e. single, shared etc.
 - Maximum and minimum price range.
 - Number of rooms.

Accommodation Listings

15. The system shall allow users to click on a listing to view detailed information including description, amenities, photos, location on a map, rent breakdown, and availability.
16. The system shall allow users to apply to a listing from the listing detail page.

Group Application for Shared Accommodation

18. The system shall allow students to form a group by adding up to 6 other students in the system.
 - 18.1. The system shall allow students to accept or reject group invitations.
19. The system shall allow students to apply as a group to any rental accommodation that has available rooms for students in a group.

Communication

20. The system shall allow students to apply to an accommodation, in group or individually, by messaging landlords using the in-app messaging app.

21. The system shall have a “My Messages” section for both student and landlord to manage chats.

Priority 2:

Accommodation Listings

16. The system shall suggest additional listings that match the user’s previous search preferences or saved listings.
17. The system shall allow users to compare multiple listings side-by-side allowing them to easily compare prices, discounts, and amenities.
18. The system shall allow users to view the distance of the accommodation from important Landmarks including the city center, Fulda University of Applied Sciences, clinics, grocery stores, etc., on a map.

Profile management

9. The system shall allow students to create their personalized profile cards with information including their hobbies, interests, and characteristics, allowing landlords to view them when the student applies for a listing.

Managing Listings

22. The system must allow students to add their preferred listings to favorites for later access.

Priority 3:

Search & Filtering

14. The system shall display a map showing the locations of all listings that match the user's selected filters.
15. The system shall provide a "save search" feature that allows users to save their search criteria for future use.

Managing Listings

24. The system shall allow students to rate a listing/landlord on a scale of 1-5.
25. The system shall provide a real-time notification feature so the student is notified of any new relevant listings.
26. The system shall provide a dashboard for the landlord to view main statistics for the listing which include the total number of views in a day/week on the listing.
27. The system shall allow registered users to report:
 - 27.1. A listing if they see something inappropriate in the post.
 - 27.2. A landlord if they break the rules.
 - 27.3. A student if they break the rules.
28. The system shall notify the Moderator of the reported listings, landlords, and students to take an action.
29. The system shall provide an interactive 3D view of the accommodation, allowing users to rotate, zoom, and pan a 3D render.

Communication

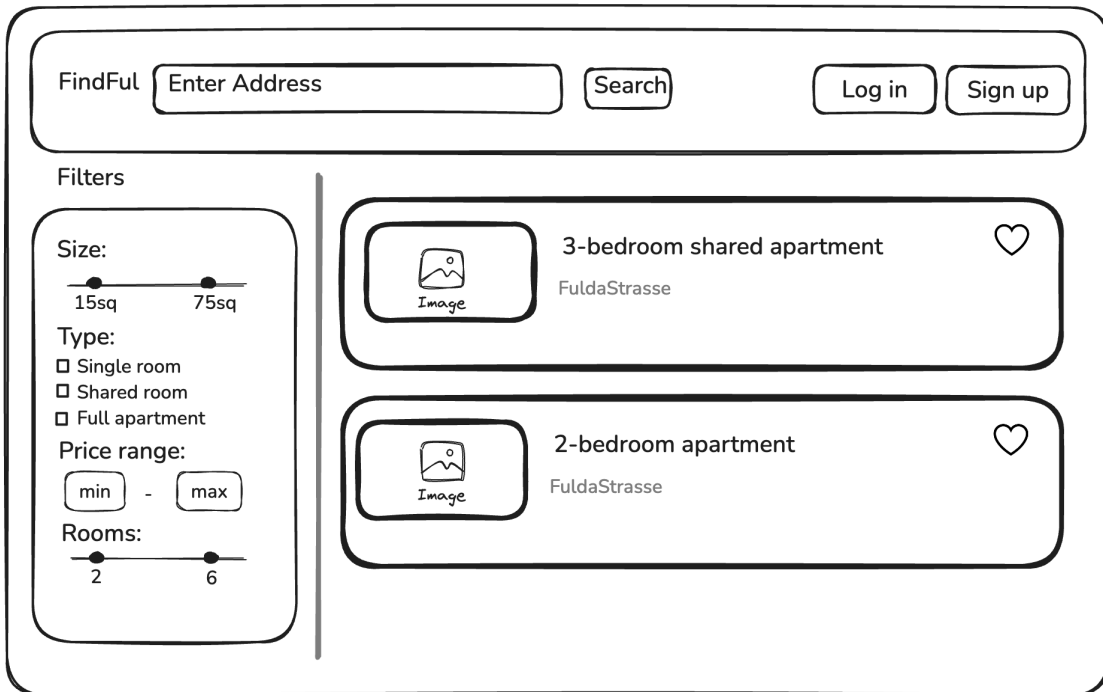
30. The system shall allow in-app messaging between two students.
31. The system shall allow users to share files, images, and videos through chat.
32. The system shall allow users to move chats from the list of active chats to an archive.

2. List of main data items and entities (expand as necessary):

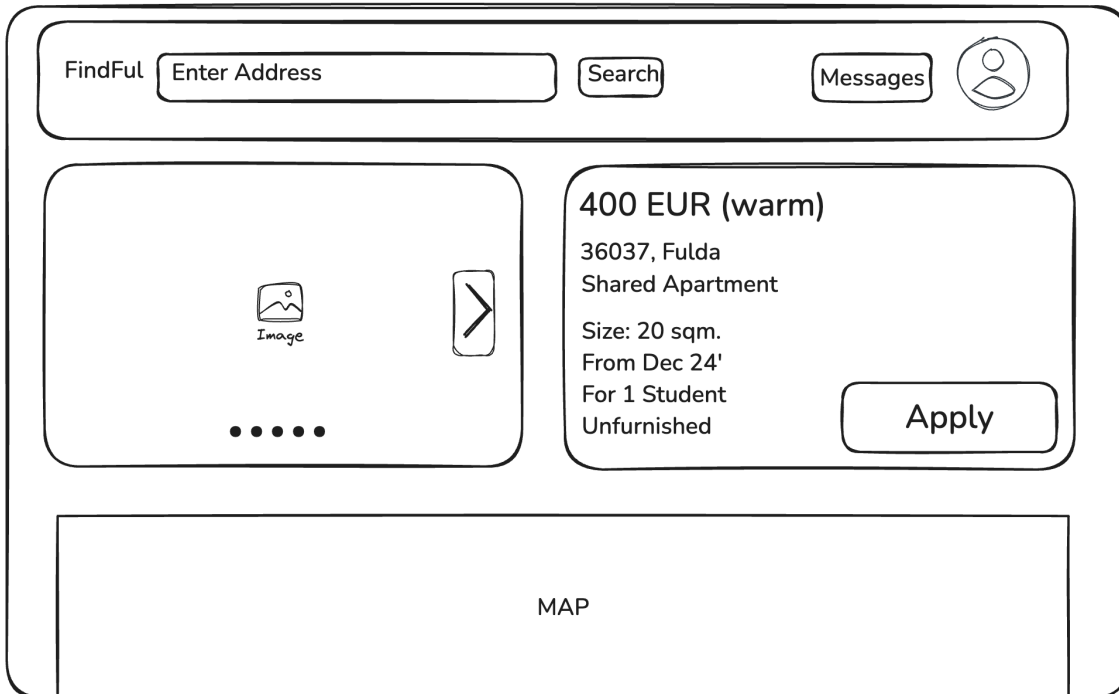
1. **Student** represents a verified student of the Fulda University of Applied Sciences. A student must complete email verification (i.e register with a hs-fulde.de email) to access the platform.
2. **Landlord** is any individual or organization offering rental properties on the platform. There is no email verification for a landlord and they can use any email address to register.
3. **Guest** is an unauthenticated visitor with limited platform access. They can browse public listings, view basic property information, and see location details, but cannot contact landlords or access full listing features.
4. **Moderator** is a system maintainer with comprehensive moderation powers. They review all new property listings and associated media, with the authority to approve or reject content based on platform guidelines.
5. **Real estate** is a representation of physical property available for rental, specifically targeting student accommodation in Fulda.
6. **Listing** is the central data element representing a property available for rent. It contains all property details including title, description, monthly rent, and property type.
7. **Student Group** is when a single student creates a group with a few other students to apply to an accommodation together.
8. **Media Item** is visual or image content associated with a rental listing, serving as a critical representation of the property's physical appearance and characteristics.

3. NEW: UI Mockups and Storyboards (high level only):

Use Case 1: Search for Housing - Tim (Guest)



Use Case 2: View Selected Housing - Ram (Student)



Use Case 3: Connect with Landlords - Ram (Student)

FindFul

Search



Mr. Muller

Ms. Sandy

Mr. Tim

3 Room shared Bedroom Apartment....

Hallo, I like the apartment location.....

Sounds good!

Use Case 4.1: Apply as a Group - Ram (Student)

FindFul

Search

Messages



Apply for

☐ Apply as an individual

☒ Apply as a group

Select a group



Message:

Use Case 4.2: Apply as a Group - Create Group

FindFul

Search

Messages



Apply for

☐ Apply as an individual

Create a new group

Members: x x

Add user by username:

Create

Apply

Use Case 5.1: Add Listing of New Real Estate - Thomas (Landlord)

FindFul

Search

msg

+



Filters

Size:

15sq 75sq

Type:
☐ Single room
☐ Shared room
☐ Full apartment

Price range:
 -

Rooms:

2 6


Image

3-bedroom shared apartment
FuldaStrasse




Image

2-bedroom apartment
FuldaStrasse

Use Case 5.2: Add Listing Form

FindFul

Search

msg

+

Details

Title

Price

 EUR

Size

 Sq m.

Rooms available

Available from

Type of apartment

Choose one

▼

Single room

Single apartment

Shared room

Description

Images

clear

Location Details

Zip code

Street/ No.
(optional)

Add

Use Case 6.1: Reviewing New Listings - Anna (Moderator)

FindFul

LISTINGS

Review

Approves

Rejected

Image

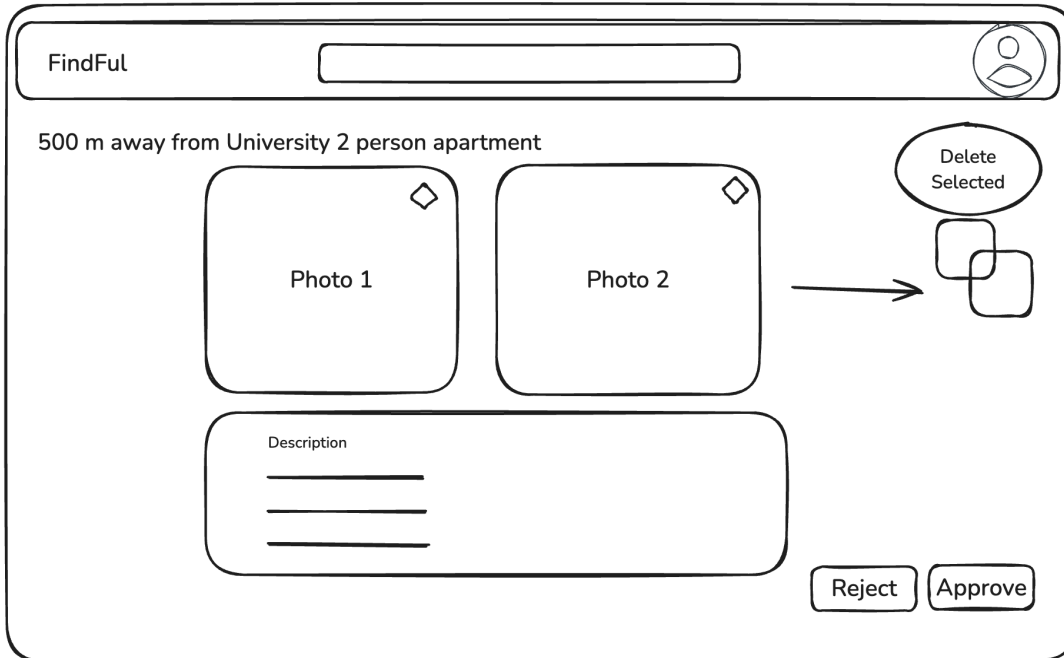
18sqm Apartment near University

Image

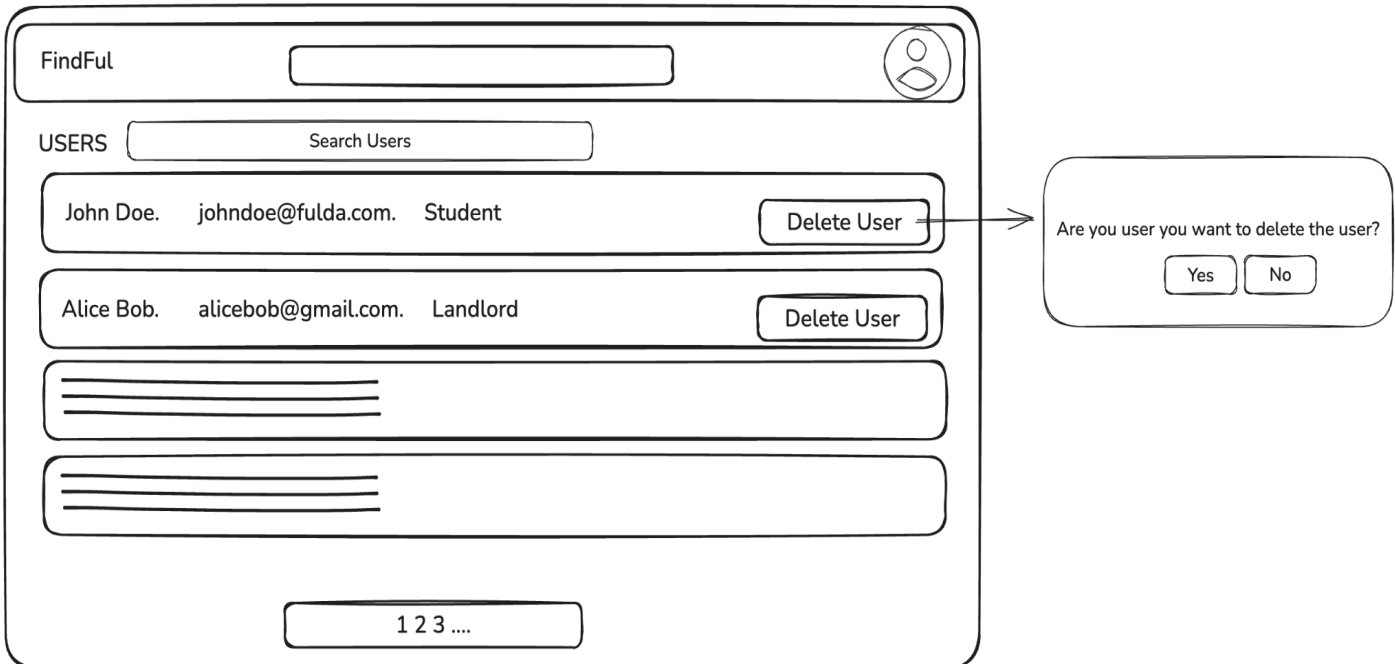
3-person shared flat in Fulda

1 2 3

Use Case 6.2: Delete Inappropriate Listing/photo

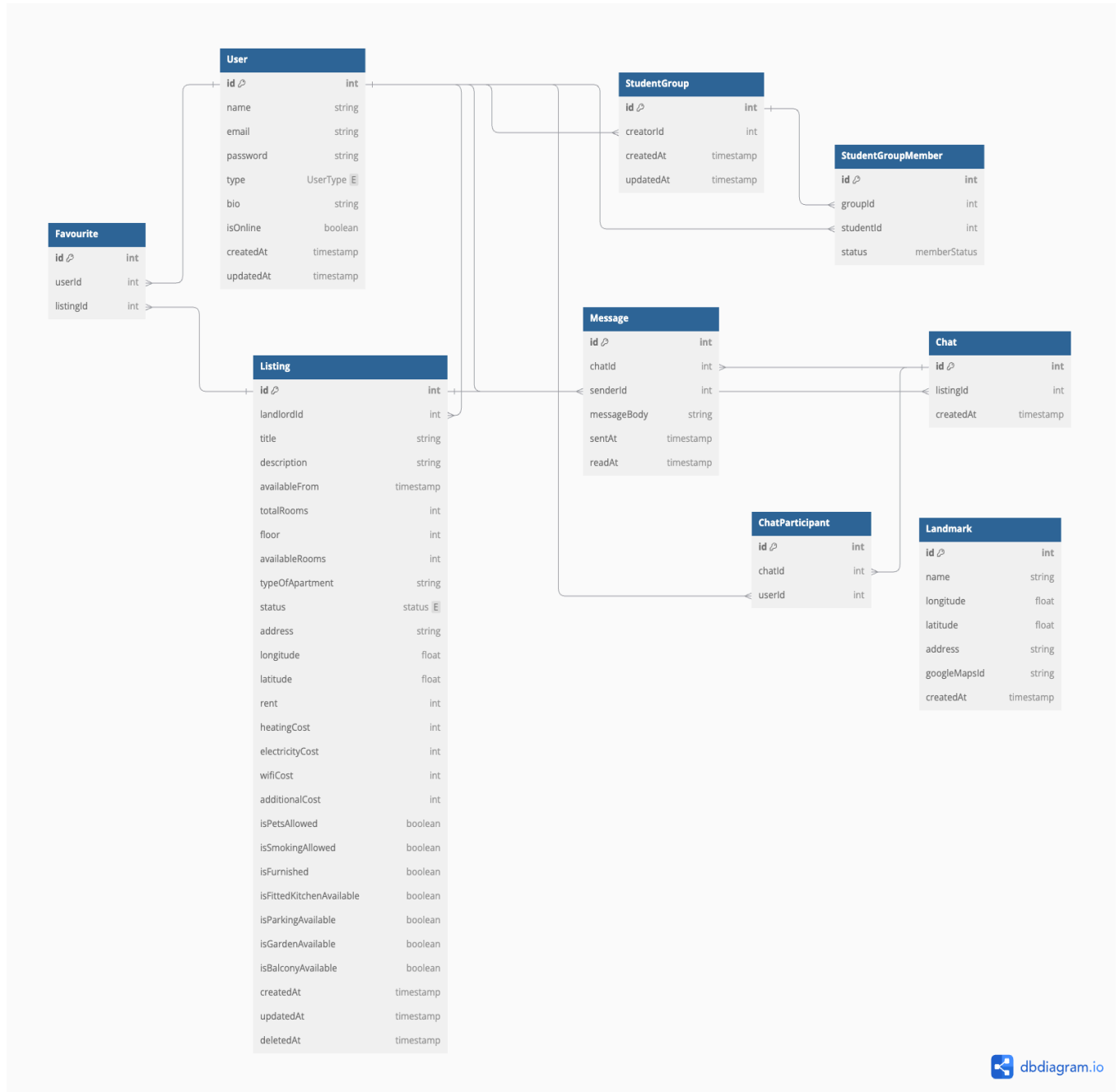


Use Case 6.3: Delete Inappropriate Users



4. NEW High level Architecture, Database Organization:

Database Schema:



The FindFul Database consists of the following main tables:

1. **User:** The User table consists of three types of users (STUDENT, LANDLORD, MODERATOR), along with user email, password (stored as a hashed string), bio, and isOnline.
2. **Listing:** The Listing table is created when a User (LANDLORD) creates a new listing for the moderator to review. It consists of a landlordId (referenced from the Users table) along with other attributes like address, zip code, rent, description, date available from, totalRooms, availableRooms, typeOfApartment, etc. It also consists of some key amenities stored as booleans, like isPetsAllowed, IsKitchenFitted, isFurnished, etc.
3. **StudentGroup:** In order to apply for accommodation as a group, a student can create a group by inviting other students in the system. The table includes a creatorId (referenced from the Users table).
4. **StudentGroupMember:** When a student is sent an invitation to join a group, their record is added to this table with a status (ACCEPTED, PENDING, REJECTED) along with the StudentId (referenced from the Users table) and the groupId (referenced from the StudentGroup table).
5. **Favourite:** Students and Landlords can favorite their desirable listings for the purpose of viewing later. The table stores the listingId (referenced from the listings table) and the userId (referenced from the Users table).
6. **Landmark:** This table has prefilled data of all the nearest landmarks a student would want to have near the apartment, i.e., University, grocery stores, clinics, city center, etc. We store the longitude and latitude for the distance to be calculated later by the Maps API.
7. **Chat:** Whenever a student applies to a listing by sending a message to the landlord, a chat is created with a primary id, and listingId.
8. **ChatParticipants:** For the purpose of one chat having more than two participants, we use this table, which will have the participantId (STUDENT or LANDLORD) and the chatId (referenced from the Chat table).
9. **Messages:** This table stores the message body, along with senderId (foreign to the User table), and chatId (foreign to the Chat table).

Media Storage:

The user and apartment photos will be stored in an S3 bucket in JPEG format. JPEG is the best choice due to its lossy compression, which is optimal for photos. It provides a good balance between quality and file size. Since being able to view apartment photos is crucial for this application, they need to be of good quality, and JPEG should fulfill that requirement effectively. Additionally, JPEG files are smaller in size compared to PNGs.

The photos can be stored in the S3 bucket in respective folders according to the database tables.

1. User Photos:

- a. **Storage Location:** users/
- b. **File Name Convention:** User photos will be stored in the user folder with the user ID as the filename.
- c. **Example:** users/<user_id>.jpeg

2. Apartment Listing photos:

- a. **Storage Location:** listings/
- b. **File Name Convention:** Photos of listings will be stored under the listings folder. This folder will contain subfolders named with the GUID of the listing from the database table, and multiple photos of the listing will be stored within the corresponding subfolder.
- c. **Example:** listings/<listing_id>/<photo_name>.jpeg

3. Thumbnails for Listing:

- a. **Storage Location:** listings/<listing_id>/thumbnails/
- b. **File Name Convention:** The thumbnails for the listing will be compressed and stored in the folder associated with the listing, using the listing_id under the /thumbnails folder, so they can be retrieved easily.
- c. **Example:** listings/<listing_id>/thumbnails/<photo_name>.jpeg

Search/filter architecture and implementation:

In FindFul, filtering is used to refine a list of listings that is presented to the user, by applying filters chosen by the user. The filtering is implemented with vanilla MySQL. Filtering with MySQL queries instead of JavaScript or TypeScript has the advantage of requiring less data to be stored. It is faster since the data needs to be fetched only once. All filters that can be applied by the user are precomposed filters that directly relate to fields in the database.

API Structure:

We will implement custom APIs to support the application's functionality, providing endpoints for user management (/users), real-estate listings (/listings), search and filtering (/search), and group management (/groups). The /users endpoint will support GET, POST, PUT, and DELETE requests, enabling functionalities such as user registration, authentication, authorization, and updating or deleting user information.

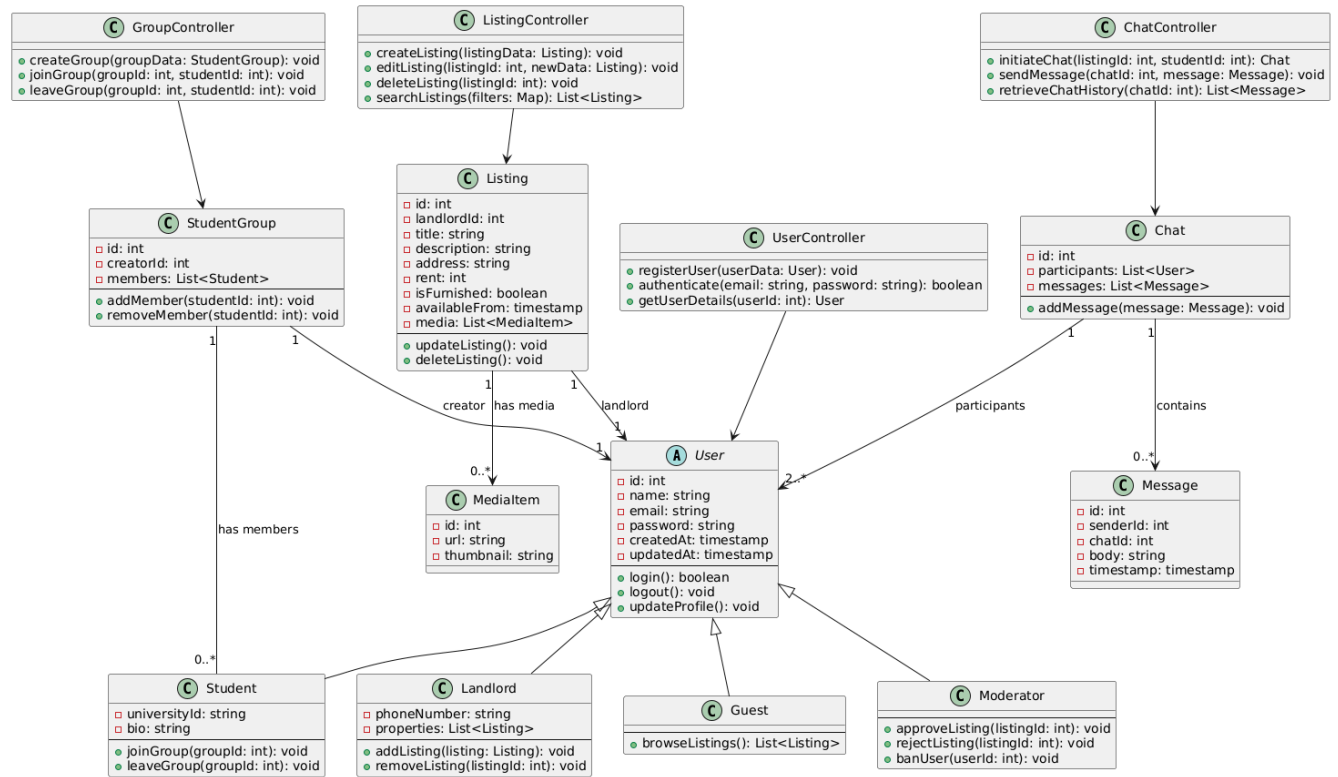
The /listings endpoint will allow for adding new apartments, updating apartment details, retrieving apartment information, and deleting listings.

The /search endpoint will handle search requests and apply filters to provide appropriate listing responses.

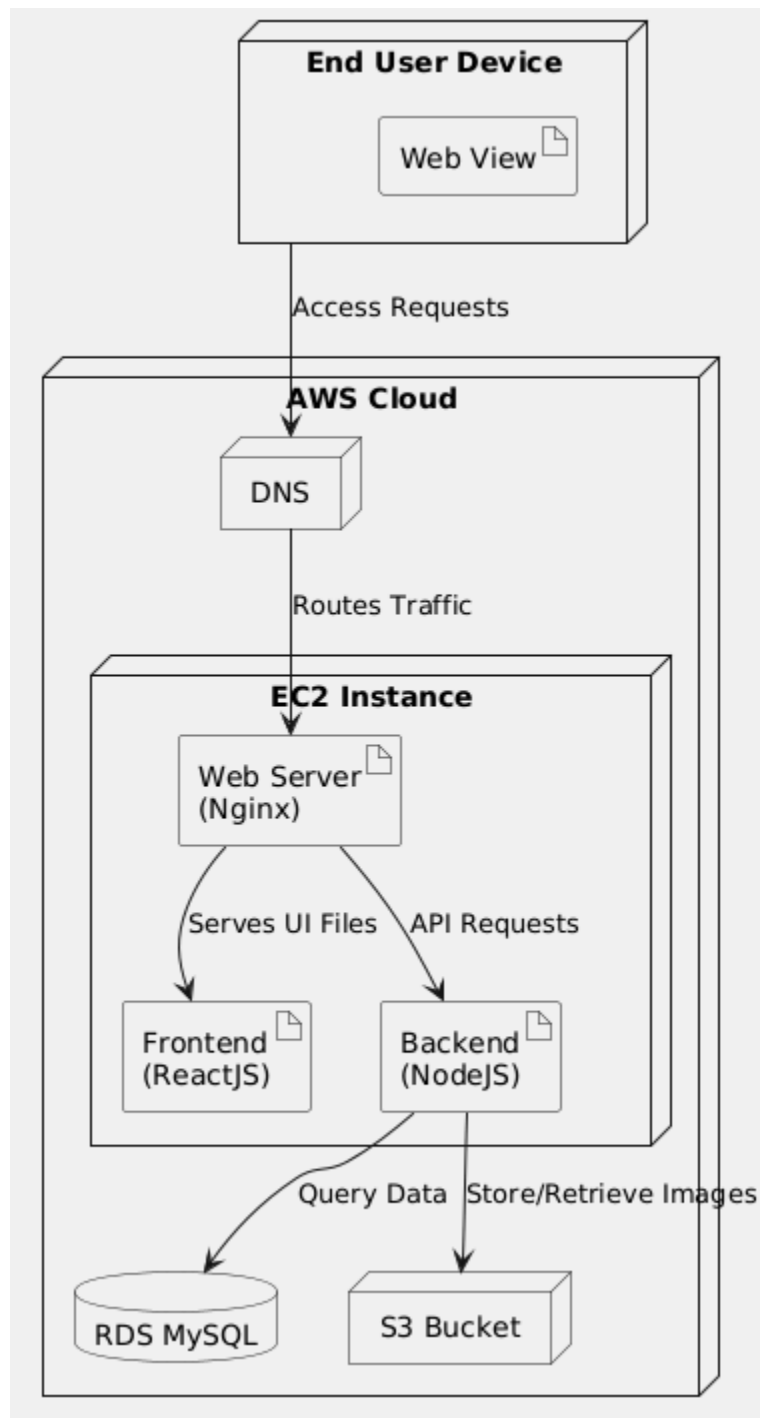
Additionally, the /groups endpoint will manage the creation, deletion, and assignment of groups and their members.

5. High Level UML Diagrams:

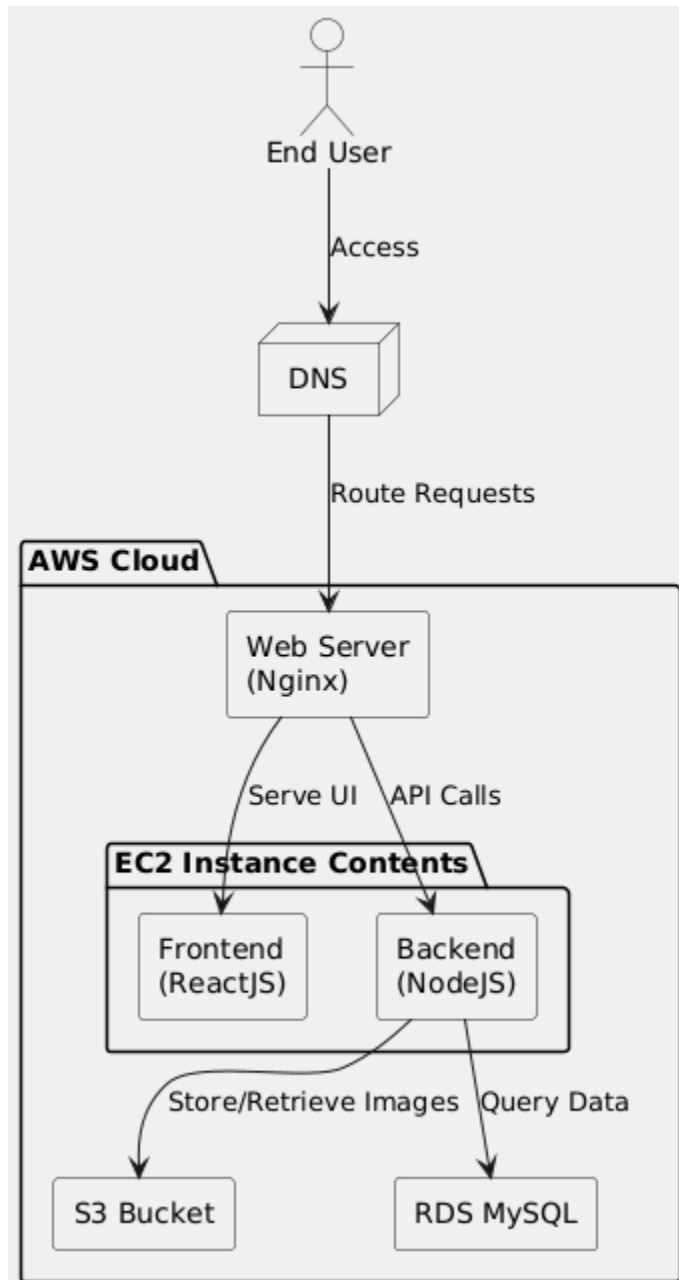
a) UML class diagram



b) UML Deployment diagram



c) Component diagram



6. Identify actual key risks for your project at this time:

1. Skill risks:

1.1 Risk:

Our team does not have a team member with a strong background in frontend development. This lack of skills poses a risk for our project since the frontend is a significant part of it.

How we plan to overcome it:

To address this, we have chosen a frontend lead who plans to continue researching our chosen frameworks, modern frontend methodologies, and best practices. The other team members will assist the frontend lead with their prior knowledge or newly acquired knowledge when needed.

1.2 Risk:

The technical skill levels of our team members vary, as some team members have extensive prior knowledge of software development while others do not. This may challenge the task distribution and communication among other team members.

How we plan to overcome it:

The task distribution is done with the skill levels of the team members in mind. If more knowledge is needed to complete a task, the team member will research it and also receive support from others. The team regularly discusses the task distribution, and if beneficial, task solutions are presented.

2. Schedule risks:

2.1 Risk:

Our weekly internal team meeting on campus takes place on Monday morning. If someone is unable to participate in it, we unfortunately do not have another time to meet up in person.

How we plan to overcome it:

So far, it has proven to be a good alternative to meet up online in the evenings. These online meetings are also used to explain new technologies to the team. If an in-person meeting is unavoidable and someone cannot attend, the team will hold the meeting and update the absent team members with a brief protocol of the meeting.

2.2 Risk:

Another scheduling risk is the amount of time we have during the semester and the examination phase. Some team members are taking multiple exams in February, which results in less time for the project during that period.

How we plan to overcome it:

To achieve good results for both the FindFul project and the exams, we will adjust our project timelines to complete the most important and critical features at the start, leaving us with smaller features close to the examination phase. We will also keep the task distribution flexible, meaning team members with other educational commitments in a week can contribute less during that week and make up for it in the coming weeks.

3. Legal Risk

3.1 Risk

The FindFul platform uses images to present the different accommodations with listings to guests and students. This requires a lot of images that must be free to use.

How we plan to overcome it:

The team intends to use copyright-free images from websites like Pixabay, Pexels, or Free Images. These websites offer a wide range of images that are free to use for non-commercial projects. For each image, the license will be checked for a correct usage.

7. Project management:

For our team project management, we are using Trello. With Trello, our team is able to create different tasks, assign them to team members and Trello helps the team to get an overview of the project.

We divided our Trello board into six sections. The first section is “To-Do,” in which we collect future tasks. Once a team member actively works on a task, the task is moved to the section “Doing.” If the team member has finished the task, they put it in the section “Review” for other team members to review it. Tasks that are ready for production are placed in “Done.” In the section “Resources,” we collect all outside resources that we are using to be able to access them quickly. Besides links to our milestone documents or our GitHub rules, we collect links to tutorials that concern every team member. Lastly, we use the section “Weekly Meeting Notes,” where we put notes about our meetings and the To-Dos that developed from them.

Trello automatically adds an ID to the tasks. We use this ID in our Git commit messages for a better overview of what the code change was about.

Master Team Project WS2024

FindFul

Team 1

Member Name	Role
Sarah Majeed (sarah.majeed@informatik.hs-fulda.de)	Team Lead and Document Master
Samrat Bikram Shah	Backend Lead and Github Master
Annika Kristin Kaul	Frontend Lead
Kemer Jimenez	Team Member
Abdul Rahman Zuhaib	Team Member

Milestone 4

23/01/2024

Date Submitted	23/01/2024
----------------	------------

1. Product summary:

Product Name: FindFul

Major Functions:

1. Students and landlords can create an account by adding their email address, password, and username.
2. Students will be verified with their emails to have the domain “@informatik.hs-fulda.de” to ensure that they’re students of Fulda University of Applied Sciences before registration is successful.
3. Registered and Unregistered users can access our search section to perform searches using the address and postal code of the rental listing.
4. They can filter by the apartment type, size of the apartment, no. of rooms, amenities, and price range.
5. Landlords can create detailed advertisements of their listing by adding the property specifications like no. of rooms, size, amenities etc.
6. Landlords can add images to their listing for visual clarity.
7. Findful has an admin dashboard so any listings added by the landlord will first be approved by an admin before it is accessible to other users.
8. The admin can remove/ban users from the platform.
9. Students can view detailed information of a listing and can apply, either as an individual or as a group of students.
10. The students will have the option to choose from their existing groups on the app or create a new one by inviting other students.
11. On applying individually, the student can do real-time chat with the landlord of that listing.
12. On applying as a group, the group of users can have real-time chat with the landlord of that listing.

URL: <https://findful.us.to/>

2. Usability test plan:

2.1 Test objectives:

The tested function is the listing search on the FindFul homepage, including the search filters. The primary objective of the usability test is to evaluate the functionality, user experience and overall satisfaction with the search bar and filters. It is assessed how easily users can locate and interact with the search bar. The intuitivity of filter-application is tested. It is also tested how well users can interpret the results of the search.

The search is the most important function of FindFul, as it is the main navigation to specific listings. Additionally the search is embedded in FindFul's homepage. For a good first impression of FindFul, the homepage should work flawlessly to convince the user to stay on the website.

The findings from this usability test will directly influence future iterations of the search functionality, ensuring it aligns with user expectations and needs while maintaining its role as a critical component of the FindFul platform.

2.2 Test background and setup:

System setup:

The test will be conducted on a Mac with macOS 14.6, using Google Chrome.

Starting point:

At the beginning of the test, the user will be on the homepage of FindFul. No search or search filters are applied prior to the usability test. The user is not logged in.

Intended users:

The intended users of FindFul are guests who want to browse student housings in Fulda, students of the Fulda University of Applied Science looking for housing in Fulda, Landlords who want to distribute their listings and look at opponent listings, as well as Moderators of FindFul who want to make sure all listings follow FindFul's rules.

URL of the system to be tested:

<https://findful.us.to/>

What is to be measured:

The test focuses on evaluating user satisfaction, based on the ease of locating and using the search bar, usability and clarity of the filters and perceived relevance of the search results.

Usability Task description:**Task:**

Find out how many listings are available on the Leipziger Straße that cost less than 300€ warm rent and have a balcony.

User satisfaction: After completing the task, users will be asked to rate 3 statements using a 5-point Likert scale from 1: strongly disagree to 5: strongly agree:

Strongly disagree	Disagree	Neutral	Agree	Strongly agree
☐	☐	☐	☐	☐

Results:**Measurement of effectiveness:**

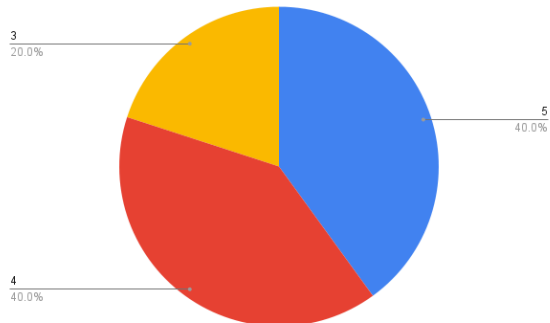
	User 1	User 2	User 3	User 4	User 5	Total
User states they found 1 listing	X	X	✓	✓	✓	3/5
The search bar and filters were used	✓	✓	✓	✓	✓	5/5
The user did not need assistance to solve the task	✓	X	✓	✓	X	3/5

Measurement of efficiency:

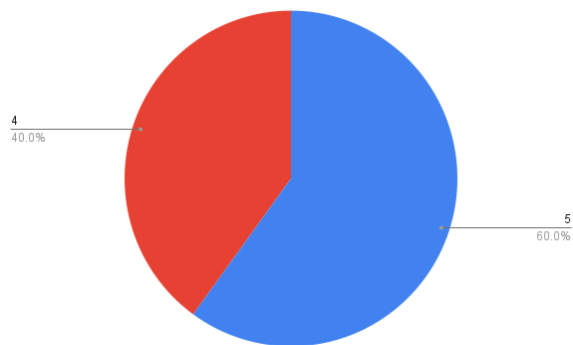
	User 1	User 2	User 3	User 4	User 5	Total
The user finishes the task in less than one minute	✓	X	X	X	X	1/5
The user needs less than 10 clicks to solve the task	✓	X	✓	✓	X	3/5

Likert scale questions:

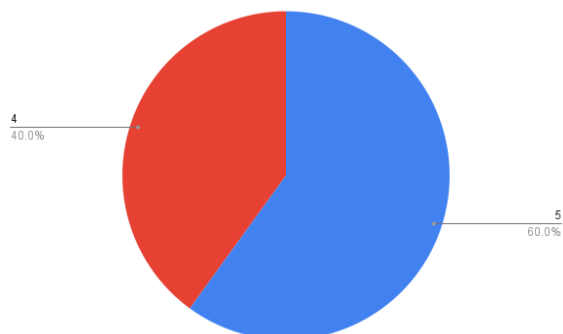
Q1: The search bar was easy to locate and use.



Q2: The use of filters was intuitive.



Q3: The design and layout of the search results page were visually appealing and helped me quickly find the information I needed.



3. QA test plan:

The objective of this test plan is to test the possible scenarios of the search functionality. These can help us conclude that the search content and their filters are working as expected. We are also checking the validation cases such as special characters not being allowed.

System setup:

The tests are conducted on a Mac with macOS 14.6, using Google Chrome. The url for the content to be tested can be found at: <https://findful.us.to/>

Test No.	Test Title	Test Description	Test Input	Expected Output	Results
1	Search Bar: By Address and Postal Code.	The search bar should search by address. For instance, if the address is Leipziger, typing “Le” in the search bar and searching should also display the result. Same logic for postal code.	“Street” or 36039	All results that have address “Street” or Postal Code 36039	Pass
2	Search Dropdown Filters: Apartment Type	The results should be filtered based on what apartment type is chosen and applied in the filter.	Single Room Apartment	All results that are single room apartments should be displayed.	Pass
3	Search Tag Filters: Amenities	Here, multiple amenities can be selected and filtered out. Based on what Amenities are selected, the results can be filtered out.	Parking and Fitted Kitchen	All results with Fitted Kitchen and Parking must be displayed.	Pass
4	Search Range	Price range of	Size: 0-40	All results	Pass

	Filters: Price, Size, Rooms	apartments, size and number of rooms available can be managed and be filtered out.	Rooms: 0-1 Price Range: 100-350	having values in the following ranges must be displayed.	
5	Search Filter: Distance from University	The distance from University is calculated based on what location the user puts in. The user can enter how many kilometers they want the accommodation to be in.	5	All results within 5 km distance of the university shall be displayed.	Pass
6	Search Box: Special Character	The Search Box should not allow special characters such as % which can potentially cause security risks.	%^	The search should not be possible and there must be a validation message displayed.	Pass

4. Code Review:

Sarah reviewed Zuhaib:

 sarah-gsd reviewed 2 weeks ago View reviewed changes

frontend/findful/src/pages/profile/EditProfilePage.jsx Outdated ⚙️ Hide resolved

```
...    @@ -65,6 +79,7 @@ const EditProfilePage = () => {
65    79      await updateUserProfile(userId, updatedValues);
66    80      message.success("Profile updated successfully!");
67    81      fetchUserProfile(userId);
82    +    navigate("/");
```

 sarah-gsd 2 weeks ago

On form submit, consider reloading the page so the user can see the changes rather than redirecting to homepage.

You can go so by replacing navigate with the following:
window.location.reload();



 Reply...

Unresolve conversation sarah-gsd marked this conversation as resolved.

 sarah-gsd reviewed 2 weeks ago View reviewed changes

frontend/findful/src/pages/profile/EditProfilePage.jsx Outdated ⚙️ Hide resolved

```
...    @@ -51,6 +53,18 @@ const EditProfilePage = () => {
51    53      setPhoneError(value.length < 7 ? "Phone number must have at least seven digits." : "");
52    54    };
53    55
56    +    const handleAge = (_, value) => {
57    +      return new Promise((resolve, reject) => {
58    +        if (value === undefined || value === null || value === "") {
```

 sarah-gsd 2 weeks ago · edited

This if statement is not required as we do not need to make adding age mandatory. Currently, all fields are required while editing the profile data which should not be the case.

You can change the schema as follows:

model Profile {
 id Int @id @default(autoincrement())
 age Int?
 gender String?
 nationality String?
 phone String? @unique
 bio String?
 user User @relation(fields: [userId], references: [id])
 userId Int @unique
}

Note that by adding a "?" after the attribute type, we can make the field optional.
Also note, after updating the schema, run "npx prisma migrate dev" in your backend terminal so the changes are reflected in your db.
Read -> <https://www.prisma.io/docs/orm/prisma-migrate/getting-started>



 Reply...

Unresolve conversation sarah-gsd marked this conversation as resolved.



sarah-gsd requested changes 2 weeks ago

[View reviewed changes](#)

sarah-gsd left a comment

...

I have added some comments for small changes.



ARZuhaib added 3 commits [last week](#)



Merge branch 'master' of <https://github.com/samratbs-fulda/gdsd> into ...

597aff0



#77: Reviewed comments for profile page

a0caef5



Merge branch 'master' of <https://github.com/samratbs-fulda/gdsd> into ...

672b47c



sarah-gsd reviewed last week

[View reviewed changes](#)

frontend/fulful/src/pages/profile/EditProfilePage.jsx Outdated

99	-	<Form.Item label="Age" name="age">
100	-	<Input type="number" />
98	+	
99	+	<Form.Item label="Age" name="age" rules={[{ validator: handleAge }]}>



sarah-gsd last week

...

The handleAge is not defined in the file as it is removed. As a result page will crash. Consider removing the following from the file:

```
rules={[{ validator: handleAge }]}
```



Reply...

Resolve conversation

Annika Reviewed Samrat:

 annikaul requested changes last month View reviewed changes

 annikaul left a comment

The searchText and card size should be reworked before merging.
Otherwise the PR looks good!



backend/src/routes/listingRouter.js Outdated

```
59 +   }
60 +   });
61 + }
62 + if (postal_code) searchText = postal_code;
```

 annikaul last month ...
The variable searchText needs to be defined beforehand. Otherwise if the postal code is empty nothing happens in line 71 (error 500).


 Reply...

Resolve conversation

frontend/findful/src/pages/Homepage.jsx Outdated

Comment on lines 177 to 179

```
177 +           style={{ width: 380,
178 +                 marginLeft: 20
179 +               }}
```

 annikaul last month ...

frontend/findful/src/pages/Homepage.jsx Outdated

Comment on lines 177 to 179

```
177 +           style={{ width: 380,
178 +                 marginLeft: 20
179 +               }}
```

 annikaul last month ...
What is the reason for this change? It is not responsive with it and the listing cards are overlapping for a smaller screen width.


 Reply...

Resolve conversation

backend/src/routes/listingRouter.js Outdated

Comment on lines 64 to 68

```
64 + if (min_price) filters.warmRent = { gt: parseFloat(min_price) };
65 + if (max_price) filters.warmRent = { ...filters.warmRent, lt: parseFloat(max_price) };
66 + if (size) filters.size = { gt: parseInt(size[0]), lt: parseInt(size[1]) };
67 + if (rooms) filters.totalRooms = { gt: parseInt(rooms[0]), lt: parseInt(rooms[1]) };
68 + if (max_distance) filters.distanceFromUni = { lt: parseFloat(max_distance) };
```

 annikaul last month ...
For some filters it might make sense to use gte (greater than or equal) or lte (lower than or equal)


 Reply...

Resolve conversation

Annika reviewed Sarah:

UI: enhance chat UI #80

Merged

annikaul merged 1 commit into master from feature/groupchat last week

Conversation 1

Commits 1

Checks 0

Files changed 2

sarah-gsd commented last week

No description provided.

UI: enhance chat UI

9df3d4c

annikaul approved these changes last week

View reviewed changes

annikaul left a comment

The new design looks great!
It might make sense though to use re-usable colors with tokens by AntDesign in the future for an easier change and corporate look of the overall design of the website.

Merged

annikaul merged commit 7af1327 into master last week

Revert

Samrat Reviewed Annika:

#116 & #115 - Feature/annikaul #81

samratbs-fulda merged 5 commits into master from feature/annikaul 20 hours ago

samratbs-fulda reviewed 2 days ago

View reviewed changes

samratbs-fulda left a comment

Overall its fine. Cleanup is recommended

frontend/findful/src/components/addListingContent/AddListingForm.jsx

256 - disabledDate={(current) => current && current.isBefore(moment(), 'day')}

257 - onChange={handleAvailableFromChange}

258 - />

311 + placeholder="Select a date"

samratbs-fulda 2 days ago

Guessing these are from formatting.

Reply...

Resolve conversation

frontend/findful/src/utils/inputValidation.js

... @@ -1,5 +1,5 @@

1 // Regex for allowed characters

2 - export const allowedCharactersRegex = /^[a-zA-Z0-9\s~.!,@]*\$/;

2 + export const allowedCharactersRegex = /^[a-zA-Z0-9\s~.!,@]*\$/;

samratbs-fulda 2 days ago

Good catch. Definitely forgot about those letters

Reply...

Resolve conversation

frontend/fulcrum/src/components/addListingContent/AddListingForm.jsx

125 + format: 'json'

126 + }).toString();

127 +

128 + const url = `https://nominatim.openstreetmap.org/search?\${query}`;

samratbs-fulda 2 days ago • edited

...

Might be a good idea to delegate this call to a service or utility. Too much code in one place

Reply...

Resolve conversation

annikaul added 2 commits 20 hours ago

#114 – Add listings with real address to seed script497622d

#116 – Refactor fetching listing positionff3d4ee

samratbs-fulda commented 20 hours ago

...

The changes are too valuable to not merge. Code cleanup can happen in later PRs.

Zuhaib Reviewed Kemer:

ARZuhaib reviewed 5 minutes ago

View reviewed changes

backend/src/services/listingService.js

ARZuhaib 5 minutes ago • edited

...

I see you're using very similar code in `getListingsByStatus()` (and also in other methods like `getListingsByLandlordId()`) to loop through each listing, build a thumbnail path, attempt to fetch an image and finally fall back to a default if it fails. This duplication can introduce bugs if you ever need to change the logic (for example, the fallback image or the folder path) in multiple places.

You can consider refactoring the repeated image-fetching code into a small helper function, for example:

```
async function fetchThumbnailForListing(listingId) {
  try {
    const folderKey = `${process.env.NODE_ENV}/listings/${listingId}/thumbnails`;
    return await S3Service.fetchAllImages(folderKey, { multiple: false });
  } catch (error) {
    console.error('Error fetching image for listing ${listingId}:', error);
    return await S3Service.fetchImage("image.webp");
  }
}
```

Then, inside `getListingsByStatus()` (and other methods), you can do:

```
const listingsWithImages = await Promise.all(listings.map(async (listing) => {
  const image = await fetchThumbnailForListing(listing.id);
  return { ...listing, img: image };
})));
```

This makes your code more maintainable and easier to read. If you need to change your fallback image, you only update the helper function.

Reply...



ARZuhaib reviewed 15 minutes ago

[View reviewed changes](#)

frontend/findful/src/pages/Login/Register.jsx

 ARZuhaib 15 minutes ago

Good job on improving the password security. Just a suggestion, consider adding a small inline help tooltip or strength indicator to guide the user while typing.



 Reply...

Kemer reviewed Zuhaib:



ARZuhaib commented 2 weeks ago

[View reviewed changes](#)

No description provided.



 ARZuhaib added 6 commits 3 weeks ago

 minor bug fix 018765c

 minor bug fix cb1f236

 Merge branch 'master' of https://github.com/samratbs-fulda/gdsd into ... 72198d7

 #94: Functioning profile edit page 480f751

 merge conflicts e210737

 profileService with debugging ba156e3

 ARZuhaib requested a review from Kemer-Jimenez 2 weeks ago



 Kemer-Jimenez approved these changes 2 weeks ago

[View reviewed changes](#)

Kemer-Jimenez left a comment

Profile page get and patch working. Add redirection when changes are saved. Clean the unused profile code in the registration service.



 Kemer-Jimenez merged commit 0988937 into master 2 weeks ago

[Revert](#)

Conversation 1

Commits 2

Checks 0

Files changed 2



ARZuhaib commented on Dec 18, 2024

[View reviewed changes](#)

No description provided.



 ARZuhaib added 2 commits 2 months ago

 Add profile model and update seed script 7815f76

 Merge branch 'master' of https://github.com/samratbs-fulda/gdsd into ... 56fcbb8

 ARZuhaib requested review from Kemer-Jimenez and sarah-gsd 2 months ago

 ARZuhaib closed this on Dec 18, 2024



 Kemer-Jimenez requested changes on Dec 18, 2024

[View reviewed changes](#)

Kemer-Jimenez left a comment

Add profile reference in the user model.
Add userid in the profile data



5. Self-check on best practices for security:

As for the development of this full stack product, we ensure the use of the best practices for security in our three main components: Frontend, Backend and Cloud.

Frontend:

- **Dom manipulation:** Avoid direct DOM manipulation with react's virtual Dom to prevent injection attacks.
- **User inputs:** Always sanitize inputs before sending requests to the backend.
- **Https:** Always use ssl/tls for communication.

Backend:

- **RBAC (role based access control):** The access to the content is restricted depending on the user role that can be GUEST, STUDENT, LANDLORD, MODERATOR.
- **Secure Auth:** Jwt are implemented with expiration, to handle role authorization.
- **Password:** Passwords are hashed using bcrypt with salt. Passwords must be 12 characters long, contain uppercase, lowercase, numbers and special characters.
- **Environment variables:** Secrets like password salt, aws credentials and database url are stored in .env files.
- **CORS policy:** Backend service only can respond to requests from our domain.
- **ORM:** To prevent sql injections, we use prisma ORM.
- **Dependencies:** Regularly run npm audit to keep dependencies updated.

Cloud:

- **IAM roles:** Backend services communicate to other services in aws using IAM users instead to access keys to enhance security.
- **SSH:** Use secure shell protocol to connect and only with key-based authentication.
- **VCP:** Our aws database service (RDS) is not publicly accessible. It is only accessible through our private subnet.
- **Security groups:** RDS allows connections only from our ec2 instances.
- **S3 privacy:** S3 is private. Public access is denied at any instance.
- **S3 IAM:** Read and write rights are given through an IAM user.
- **Pre-signed urls:** Temporary urls are generated to give access to s3 contents.
- **Nginx for DDoS:** Nginx has an inherent protection to DDoS attacks thanks to its event-driven architecture.
- **SSL:** Ensure a secure and private communication between user and services.

6. Self-check: Adherence to original Non-functional specs – performed by team leads:

1. Application shall be developed, tested and deployed using tools and servers approved by Class CTO and as agreed in Milestone 0. Application delivery shall be from chosen cloud server. **DONE**
2. Application shall be optimized for standard desktop/laptop browsers e.g. must render correctly on the two latest versions of two major browsers. **ON TRACK**
3. All or selected application functions must render well on mobile devices. **ON TRACK**
4. Data shall be stored in the database on the team's deployment cloud server. **DONE**
5. Full resolution free media shall be downloadable directly, and full resolution media for selling shall be obtained after contacting the seller/owner. **DONE**
6. No more than 50 concurrent users shall be accessing the application at any time. **ON TRACK**
7. Privacy of users shall be protected, and all privacy policies will be appropriately communicated to the users. **ON TRACK**
8. The language used shall be English (no localization needed). **DONE**
9. Application shall be very easy to use and intuitive. **DONE**
10. Application should follow established architecture patterns. **DONE**
11. Application code and its repository shall be easy to inspect and maintain. **ON TRACK**
12. Google analytics shall be used (optional for Fulda teams). **NA**
13. No e-mail clients shall be allowed. **DONE**
14. Pay functionality, if any (e.g. paying for goods and services) shall not be implemented nor simulated in UI. **DONE**
15. Site security: basic best practices shall be applied (as covered in the class) for main data items. **DONE**
16. Application shall be media rich (images, video etc.). Media formats shall be standard as used in the market today. **ON TRACK**
17. Modern SE processes and practices shall be used as specified in the class, including collaborative and continuous SW development. **ON TRACK**
18. For code development and management, as well as documentation like formal milestones required in the class, each team shall use their own GitHub to be set-up by class instructors and started by each team during Milestone 0. **DONE**
19. The application UI (WWW and mobile) shall prominently display the following exact text on all pages "Fulda University of Applied Sciences Software Engineering Project, Fall 2024 For Demonstration Only" at the top of the WWW page. (Important to not confuse this with a real application). **DONE**



Filters

Apartment Type

Single-room apartment

Furnished Status

All

Price Range

0 1500

Size (sq.m)



Rooms



Amenities

Fitted Kitchen

Max Distance from University (km)

5

Apply Filters

Remove Filters

Enter address or postal code

Search

Listings



SINGLE in Leipziger Straße - 20 sqm, Flo...

SINGLE

Rent: 260€
Size: 20 sq.m
Rooms Available: 2
Address: Leipziger Straße 12, 36037
Distance from University: 1.2 km

View Details



SINGLE in Magdeburger Straße - 40 sqm...

SINGLE

Rent: 430€
Size: 40 sq.m
Rooms Available: 4
Address: Magdeburger Straße 85, 36037
Distance from University: 0.5 km

View Details



SINGLE in Leipziger Straße - 60 sqm, Gr...

SINGLE

Rent: 580€
Size: 60 sq.m
Rooms Available: 2
Address: Leipziger Straße 170, 36039
Distance from University: 0.9 km

View Details



SINGLE in Leipziger Straße - 20 sqm, Floor 1



260€ (warm)

Rent

20²m

Size

2

Rooms



Conversations

SINGLE in Leipziger Straße - 20 sqm, Fl...

SUBLET in Lindenstraße - 30 sqm, Floo...

SINGLE in Magdeburger Straße - 40 sq...

SINGLE in Leipziger Straße - 20 sqm, Fl...

SUBLET in Lindenstraße - 30 sqm, Floor 2

● Logged in: student_user

Participants: student_user student_user2 landlord_user

What about the deposit and utilities?

sent by student_user2

Deposit is \$1,500, and utilities are not included.

sent by landlord_user

Yes, one parking spot is included in the rent.

sent by landlord_user

Can we visit the apartment this weekend?

sent by student_user2

Sure! Saturday afternoon works for me.

sent by landlord_user

Got it. Is there a parking spot available?

Type a message...

